```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from google.colab.patches import cv2_imshow
```

```
uploaded = files.upload()
```

Choose Files scenery.jpg

scenery.jpg(image/jpeg) - 137348 bytes, last modified: 9/10/2026 - 100% done

```
Saving scenery.jpg to scenery.jpg
```

```
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)
```

```
Uploaded file: scenery.jpg
```

```
image = cv2.imread(filename)

print("Image shape:", image.shape)
```

```
Image shape: (842, 474, 3)
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
plt.figure(figsize=(8,6))
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.show()
```




```
gaussian = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
plt.figure(figsize=(8,6))
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Blur")
plt.axis('off')
plt.show()
```




```
median = cv2.medianBlur(gray, 5)
```

```
plt.figure(figsize=(8,6))
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis('off')
plt.show()
```




```
def add_salt_pepper_noise(image, amount=0.03):
    noisy = image.copy()

    num_pixels = int(amount * image.size)

    # Salt noise
    coords = (
        np.random.randint(0, image.shape[0], num_pixels),
        np.random.randint(0, image.shape[1], num_pixels)
    )
    noisy[coords] = 255
```

```
    # Pepper noise
    coords = (
        np.random.randint(0, image.shape[0], num_pixels),
        np.random.randint(0, image.shape[1], num_pixels)
    )
    noisy[coords] = 0

    return noisy
```

```
noisy_image = add_salt_pepper_noise(gray)
```

```
plt.figure(figsize=(8,6))
plt.imshow(noisy_image, cmap='gray')
plt.title("Salt-and-Pepper Noisy Image")
plt.axis('off')
plt.show()
```




```
median_noisy = cv2.medianBlur(noisy_image, 5)
```

```
plt.figure(figsize=(8,6))
plt.imshow(median_noisy, cmap='gray')
plt.title("Median Filtered Noisy Image")
plt.axis('off')
plt.show()
```




```
average = cv2.blur(gray, (5, 5))
```

```
plt.figure(figsize=(8,6))
plt.imshow(average, cmap='gray')
plt.title("Average (Mean) Filter")
plt.axis('off')
plt.show()
```




```
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(average, cmap='gray')
plt.title("Average Filter")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Filter")
plt.axis('off')
```

```
plt.subplot(1,3,3)
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis('off')

plt.tight_layout()
plt.show()
```




```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

laplacian = cv2.convertScaleAbs(laplacian)
```

```
plt.figure(figsize=(8,6))
plt.imshow(laplacian, cmap='gray')
plt.title("Laplacian Edge Detection")
plt.axis('off')
plt.show()
```




```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
sobel_x = cv2.convertScaleAbs(sobel_x)
```

```
plt.figure(figsize=(8,6))
plt.imshow(sobel_x, cmap='gray')
plt.title("Sobel X - Horizontal Edges")
plt.axis('off')
plt.show()
```




```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel_y = cv2.convertScaleAbs(sobel_y)
```

```
plt.figure(figsize=(8,6))
plt.imshow(sobel_y, cmap='gray')
plt.title("Sobel Y - Vertical Edges")
plt.axis('off')
plt.show()
```



```
sobel_combined = cv2.magnitude(
    cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3),
    cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
)

sobel_combined = cv2.convertScaleAbs(sobel_combined)
```

```
plt.figure(figsize=(8,6))
plt.imshow(sobel_combined, cmap='gray')
plt.title("Combined Sobel Edges")
plt.axis('off')
plt.show()
```




```
plt.figure(figsize=(15,10))

plt.subplot(2,4,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(2,4,2)
plt.imshow(average, cmap='gray')
plt.title("Average Filter")
plt.axis('off')

plt.subplot(2,4,3)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Filter")
plt.axis('off')

plt.subplot(2,4,4)
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis('off')

plt.subplot(2,4,5)
plt.imshow(laplacian, cmap='gray')
plt.title("Laplacian")
plt.axis('off')

plt.subplot(2,4,6)
plt.imshow(sobel_x, cmap='gray')
plt.title("Sobel X")
plt.axis('off')

plt.subplot(2,4,7)
plt.imshow(sobel_y, cmap='gray')
plt.title("Sobel Y")
```

```
    plt.axis('off')

    plt.subplot(2,4,8)
    plt.imshow(sobel_combined, cmap='gray')
    plt.title("Combined Sobel")
    plt.axis('off')

    plt.tight_layout()
    plt.show()
```



PART B — THEORY QUESTIONS

Q1. What is spatial filtering? How is it used in digital image processing?

Answer:

Spatial filtering is an image-processing technique in which the value of a pixel is modified based on its neighboring pixels using a filter or kernel.

It is used for:

Noise reduction Image smoothing Edge detection Edge enhancement Feature extraction

Spatial filtering works directly on the pixels of an image, which is why it is called spatial-domain filtering.

Q2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Low-Pass Filter High-Pass Filter Smooths the image Enhances edges and fine details Reduces high-frequency components Emphasizes high-frequency components Used for noise reduction Used for edge detection Can blur fine details Can make edges more prominent Examples: Gaussian, Average, Median Examples: Laplacian, Sobel

The sheet describes low-pass filters primarily as noise-reduction/smoothing techniques and high-pass filters as techniques for emphasizing edges and fine details.

Q3. Compare Average Filter, Gaussian Filter, and Median Filter.

Filter Working Principle Main Application Average Replaces a pixel with the average of neighboring pixels General smoothing Gaussian Uses a Gaussian-weighted neighborhood Noise reduction and smoothing Median Replaces a pixel with the median of neighboring values Salt-and-pepper noise removal

Average filter: Simple smoothing but may blur edges.

Gaussian filter: Gives more weight to pixels near the center and generally produces smoother results.

Median filter: Uses the median rather than the average, making it highly effective against impulse noise.

Q4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Answer:

Salt-and-pepper noise consists of randomly occurring very bright and very dark pixels.

The Median Filter replaces each pixel with the median value of its neighborhood. Extreme values such as 0 and 255 therefore have less influence than they would in an averaging operation.

As a result, salt-and-pepper noise can be removed while preserving important edges relatively well.

Q5. Explain the role of convolution kernels in spatial filtering.

Answer:

A convolution kernel is a small matrix of numerical values that is moved across an image.

At each position, the kernel values are multiplied with corresponding image pixels and combined to calculate a new pixel value.

Different kernels produce different effects:

Smoothing Sharpening Edge detection Noise reduction

Thus, convolution kernels are fundamental to spatial filtering.

Q6. What is the purpose of the Sobel and Laplacian operators in edge detection?

Answer:

Sobel Operator: Calculates image intensity gradients in specific directions. Sobel X detects changes primarily in one direction, while Sobel Y detects changes in the other direction.

Laplacian Operator: Uses second-order derivatives to highlight regions where image intensity changes rapidly.

Both operators are useful for detecting edges and fine details.

Q7. Why are filtering operations considered an essential preprocessing step in computer vision?

Answer:

Filtering can improve the quality of an image before further analysis by:

Removing unwanted noise Smoothing unnecessary variations Enhancing edges Preserving important features Improving feature extraction

Therefore, filtering can make subsequent computer vision tasks more reliable. The experiment identifies filtering as an important preprocessing operation for applications such as medical imaging, surveillance, remote sensing and autonomous systems.

Q8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Answer:

Smoothing reduces noise but can also remove important edges and fine details.

If a strong smoothing filter is used, noise may be reduced significantly, but edges can become blurred.

If less smoothing is applied, edges and details are better preserved, but more noise may remain.

Therefore, the filter and its parameters must be selected according to the application.

Q9. Mention four real-world applications where spatial filtering techniques are widely used.

Answer:

Four applications are:

Medical imaging — noise reduction and enhancement of medical images. Surveillance — improving image quality and detecting important features. Remote sensing — processing satellite and aerial imagery. Autonomous systems — preparing camera images for perception and object detection.

Q10. Compare spatial domain filtering with frequency domain filtering.